EXPERIMENT NO: 9

MODEL DEPLOYMENT: REST API WITH FLASK AND CONTAINERIZATION WITH DOCKER

AIM:

To develop a RESTful API using Flask to serve a trained machine learning model, test the prediction endpoint using JSON input, and containerize the application using Docker.

ALGORITHM:

1. Import the required libraries and load the Iris dataset.
2. Train a classification model and save it using joblib.
3. Create a Flask application and load the saved model.
4. Create a POST /predict endpoint that accepts JSON input and returns a prediction.
5. Test the API locally using a JSON request.
6. Create requirements.txt and a Dockerfile.
7. Build a Docker image and run a container exposing port 5000.
8. Test the API running inside the Docker container.

CODE:

File 1: train_model.py

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import joblib

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Model Accuracy:", round(accuracy_score(y_test, pred), 3))
joblib.dump(model, "iris_model.pkl")
print("Model saved as iris_model.pkl")
```

File 2: app.py

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)
model = joblib.load("iris_model.pkl")
names = ["setosa", "versicolor", "virginica"]

@app.route("/")
def home():
    return jsonify({"message": "Iris Prediction API is running",
                    "endpoint": "POST /predict"})
```

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(silent=True)
    if not data or "features" not in data:
        return jsonify({"error": "Provide 4 numeric features"}), 400
    try:
        features = np.array(data["features"], dtype=float)
        if features.shape != (4,):
            raise ValueError
        result = int(model.predict([features])[0])
        return jsonify({"class_id": result, "predicted_class": names[result]})
    except (ValueError, TypeError):
        return jsonify({"error": "features must contain exactly 4 numbers"}), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

File 3: requirements.txt

```
Flask
scikit-learn
joblib
numpy
```

File 4: Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY iris_model.pkl .
EXPOSE 5000
CMD ["python", "app.py"]
```

API TESTING:

POST URL: http://127.0.0.1:5000/predict

Request body (JSON):

```
{
  "features": [5.1, 3.5, 1.4, 0.2]
}
curl -X POST http://127.0.0.1:5000/predict -H "Content-Type: application/json" -d
"{\"features\":[5.1,3.5,1.4,0.2]}"
```

OUTPUT:

```
Model Accuracy: 1.0
Model saved as iris_model.pkl

HTTP/1.1 200 OK
{
  "class_id": 0,
```

```
  "predicted_class": "setosa"
}
docker build -t iris-api .
docker run -p 5000:5000 iris-api

 * Running on http://127.0.0.1:5000
```

RESULT:

A RESTful API was developed using Flask to serve a trained machine learning model. The API accepts feature values in JSON format and returns the predicted class in JSON format. The application was containerized using Docker so that the model and its dependencies can run in a consistent environment.